ℹ Information about the exam

Read and follow these instructions carefully to increase your chance of getting good marks.

- This is a closed-book exam. You may use a standard English dictionary. Otherwise, **no notes, calculators, mobile phones, or other electronic devices are allowed**.
- Each question awards a number of points, as shown at the end of each question.
- You can "flag" questions (click the small flag at the top of each question) that you e.g. want to review later.
- Some questions ask for a justification of your answer; your justification needs to relate the information given in the question, the course content, and your answer.
- Some questions ask you to write code; this code needs to conform to the coding convention. You only need to write function specifications if the question text states so.
- You may use English or Swedish to answer the exam questions.
- Eva and/or Jonas will come to the main exam hall around 9:00 to answer questions (and a bit later to other exam locations).

Consider the following code:

```
const a = 10;

const b = a < 5 ? 13 : 1;

a > b ? 7 : 11;
```

1 Substitution model - result

What is the result of the program on the left?

Answer:

Maximum marks: 1

2 Substitution model - evaluation steps

Show all evaluation steps using the **substitution model**, paying particular attention to the order of computation:

Maximum marks: 1

Consider the following code:

```
// @precondition m <= n
function mystery_sum(m, n) {
    return m === n
           ? m
           : m + mystery_sum(m + 1, n);
}
```

3 Evaluating recursive functions

What is the result of `mystery_sum(2, 5)` ?

Answer:

Maximum marks: 1

4 Iterative processes

Rewrite the function so that it gives rise to an **iterative process**. Your new function should take the same (and the same number of) arguments as `mystery_sum`.

Answer:

```
1
```

Maximum marks: 2

5 Variants

Give a **variant** for the (original) function `mystery_sum` as given on the left:

Answer:

Justify your answer, i.e. why is it a valid variant?

Maximum marks: 2

6 Higher-order functions

Consider the following function definitions:

```
function plus_one(x) {
    return x + 1;
}
function trans(f) {
    return x => 2 * f(2 * x);
}
function twice(f) {
    return x => f(f(x));
}
```

What is the result of `trans(plus_one)(1)` ?

Answer:

What is the result of `trans(twice(plus_one))(1)` ?

Answer:

Maximum marks: 2

7 List principle

Consider the following two lists:

```
const a = list(1, 2, 3);
const x = list("e", list("g", "h"), "f");
```

What is the result of `head(tail(tail(x)))` ?

Answer:

What is the result of `length(pair(a, x))` ?

Answer:

Maximum marks: 2

8 List processing

Write a **full** function specification for the following function, including **at least two examples** that explain the behavior of the function.

```
function lal(lst, k, val) {
    // variant: k
    return k === 0
                ? null
            : is_null(lst)
                ? pair(val, lal(lst, k - 1, val))
            : pair(head(lst), lal(tail(lst), k - 1, val));
}
```

Your specification should have a **precondition**.

Answer:

1

Maximum marks: 3

9 List processing abstractions

Write the function `sum_even(lst)` that takes as input a list of numbers `lst` and returns the sum of all even numbers in `lst`.

For example:

```
sum_even(list(1, 3, 3, 2, 5, 8, 9, 11, 4)); // returns 14
sum_even(list());         // returns 0
sum_even(list(1, 3));     // returns 0
sum_even(list(1, 2, 3)); // returns 2
```

You **must** use the higher-order list processing functions `map` , `filter` and/or `accumulate` to implement your function (choose one or more that are appropriate). You are **not** allowed to write a recursive function or to use `for` or `while` loops.

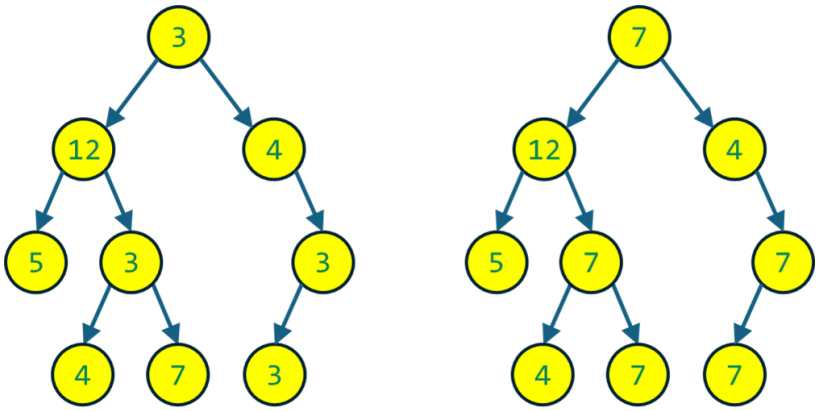
Answer:

```
1
```

Maximum marks: 3

10 Binary Tree Replace Value

Define a function `binary_tree_replace_value` that takes as parameters a binary tree `t`, a value `u`, and another value `v`. The result of `binary_tree_replace_value(t, u, v)` should be a new binary tree that is like `t` except that any node value `u` has been replaced by `v`. For example, suppose that `t` is the tree on the left in the figure, then the result of `binary_tree_replace_value(t, 3, 7)` would be the tree on the right in the figure.



Assume your program already contains

```
import { entry, is_empty_tree, is_tree, left_branch,
         make_empty_tree, make_tree, right_branch} from "binary_tree";
```

Here is a summary of the binary tree interface if you need it: Binary Tree API.

Answer:

```
1
```

Maximum marks: 3

11 Merge sort

Recall the **merge sort** sorting algorithm. Suppose the algorithm is applied to the following list:

```
list(5, 11, 2, 7, 3, 5, 15, 1)
```

What are the arguments to the immediate recursive calls of the merge sort algorithm (i.e. to the top-level `merge_sort` function)?

Answer:

Explain your answer:

Maximum marks: 2

12 Set data abstraction

At a high level, what is the difference between a **list** and a **set** data abstraction?

Do not discuss complexity or implementation details.

Answer:

Maximum marks: 1

Consider the following program:

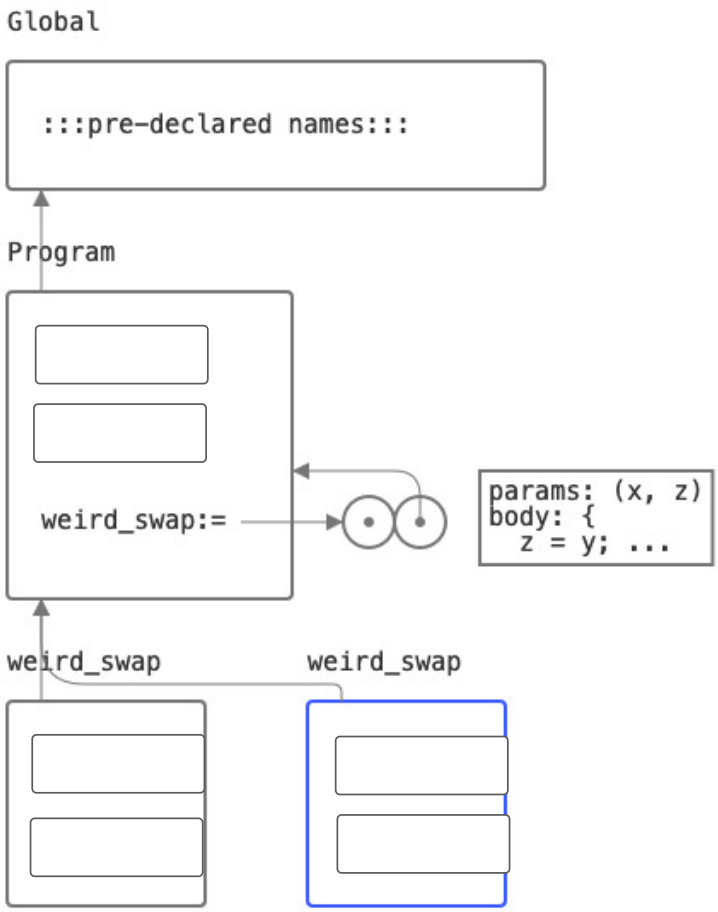
```
let x = 7;
let y = 1;

function weird_swap(x, z) {
    z = y;
    y = x;
    x = z;
}

weird_swap(3, 0);
weird_swap(5, 1);
```

13 Environment model

Fill in the missing entries in the following environment model illustration as they appear **at the end of the program**.
There are **6 blanks** in total that you should fill in. For each blank, use the format `name: value`.



Maximum marks: 3

14 Array Left Scan

Write a function `array_left_scan(f, i, a)` that takes as input a function `f`, an initial value `i`, and an array `a` of length `k` and that returns a new array `b` of length `k`, where `b[0]` is `f(i, a[0])`, `b[1]` is `f(f(i, a[0]), a[1])`, …, and `b[k-1]` is `f(f(… f(f(i, a[0]), a[1]) …, a[k-2]), a[k-1])`.

For example, the result of

```
array_left_scan((x, y) => x * y, 1, [2, 4, 6, 8, 10]
```

should be

```
[2, 8, 48, 384, 3840
```

as $1 \times 2 = 2$, $2 \times 4 = 8$, $8 \times 6 = 48$, $48 \times 8 = 384$, and $384 \times 10 = 3840$.

You may use recursion or iteration (i.e., a for or a while loop). The function should **not** change the input array `a`.

Answer:

```
1
```

Maximum marks: 3

15 Counting Change

Recall the problem of counting change from multiple lectures and SICP JS. As stated, given an amount of currency (e.g., in Swedish krona) and a set of denominations (e.g., 1 krona, 2 krona, 5 krona and 10 krona), the function `count_change` computes the number of different ways that the amount could be delivered using these denominations. For example, 5 krona could be delivered in four ways: five 1 krona, two 2 krona and two 1 krona, one 2 krona and three 1 krona, or five 1 krona.

This is the code:

```
function count_change(amount) {
    return cc(amount, 4);
}
function cc(amount, kinds_of_coins) {
    // variant: amount + kinds_of_coins + 10
    return amount === 0
           ? 1
           : amount < 0 || kinds_of_coins === 0
           ? 0
           : cc(amount, kinds_of_coins - 1) +
               cc(amount - first_denomination(kinds_of_coins), kinds_of_coins);
}
function first_denomination(kinds_of_coins) {
    return kinds_of_coins === 1 ? 1
         : kinds_of_coins === 2 ? 2
         : kinds_of_coins === 3 ? 5
         : kinds_of_coins === 4 ? 10
         : 0;
}
```

A call might look like `count_change(100)`.

Could `cc` (and thus `count_change`) benefit from **memoization**? Why? Or why not?

Answer:

Format

Words: 0

Maximum marks: 2

16 Order of Growth

Let $f(n) = n^2 + 3n - 4$

Which of the following statements are true? Select all, i.e. add a check mark for all that are true.

- [ ] $f(n) \in O(n^3)$
- [ ] $f(n) \in \Omega(n \log n)$
- [ ] $f(n) \in \Theta(n^2)$
- [ ] $f(n) \in \Omega(n^3)$

Maximum marks: 2

Muster theorem

Let $T(n)$ be a function such that for some $a, b, c, n_0 > 0$,

$$T(n) = \begin{cases} c, & \text{if } n \leq n_0; \\ aT(n-b) + f(n), & \text{if } n > n_0. \end{cases}$$

If $f(n) \in \Theta(n^d)$ with $d \geq 0$, then

$$T(n) \in \begin{cases} O(n^d a^{n/b}), & \text{if } a > 1; \\ \Theta(n^{d+1}), & \text{if } a = 1; \\ \Theta(n^d), & \text{if } a < 1. \end{cases}$$

Master Theorem

Let $T(n)$be a function such that for some $a, b, c, n_0 > 0$,

$$T(n) = \begin{cases} c, & \text{if } n \leq n_0 \\ aT(n/b) + f(n), & \text{if } n > n_0 \end{cases}$$

Let $\rho = \log_b a$. Then

$$T(n) \in \begin{cases} \Theta(n^\rho), & \text{if for some } \epsilon > 0, \text{ we have } f(n) \in O(n^{\rho-\epsilon}); \\ \Theta(n^\rho \log^{k+1} n), & \text{if for some } k \geq 0, \text{ we have } f(n) \in \Theta(n^\rho \log^k n); \\ \Theta(f(n)), & \text{if for some } \epsilon > 0, \text{ we have } f(n) \in \Omega(n^{\rho+\epsilon}) \text{ and } f \text{ is regular.} \end{cases}$$

17 Solving Recurrences

Write the following recurrence in *closed form*, i.e., without recursion.

$$T(n) = \begin{cases} 1, & \text{if } n \leq 1; \\ 7T(n-3) + n, & \text{if } n > 1. \end{cases}$$

You should use the Muster or the Master theorem to solve the recurrence (see definition on the left).

State the solution of the recurrence and **explain** how you obtained it: explain which theorem you used and why, state what values the theorem's parameters have, which case applies and how you obtain the final solution.

Answer:

Format

Words: 0

Maximum marks: 2

18 Design methodology

Suppose you want to write a program that manages Internet bookmarks. For each bookmark, you want to store a name (e.g., Uppsala University), a URL (e.g., https://www.uu.se/), and the number of times the bookmark has been accessed (e.g., 42).

Define the data type *Bookmark* as we have done in the course. Your data type must have at least one **invariant**.

Give 4 meaningful **examples**, including valid, invalid and borderline/corner cases.

Answer:

```
1
```

Maximum marks: 3

19 Streams

Consider the following program:

```
function stream_map(fun, stream) {  // same as in the lecture
    return is_null(stream)
           ? null
           : pair(fun(head(stream)),
                  () => stream_map(fun, stream_tail(stream)));
}
function numbers(i) {
    display(i);
    return i > 10
           ? null
           : pair(i, () => numbers(i + 1));
}
const doubles = stream_map(x => x * 2, numbers(1));
display("---");
head(stream_tail(doubles));
display("---");
head(stream_tail(stream_tail(doubles)));
```

Which values get printed by the **display** statements (in italic) when the program is evaluated? Give the **concrete values** in the order that they appear and also **explain your answer.**

Answer and justification:

Maximum marks: 2